

杂题选讲

sunzihang

2026 年 5 月 12 日

n 名玩家使用一枚公平的 k 面骰子（骰子各面编号为 1 到 k ，即每次投掷得到 1 到 k 中任意点数的概率均为 $\frac{1}{k}$ ）进行骰子游戏。初始时，每位玩家的得分均为零。

在单次行动中，得分最少的玩家投掷骰子，并将投掷结果加到自己的得分上。若在某一时刻有多名玩家并列得分最低，则由其中随机一名玩家行动。

当任意一名玩家累计得分达到 m 分或以上时，游戏结束。求行动次数的期望值对 $10^9 + 7$ 取模。

$1 \leq n, k, m \leq 10^6$ 。2 s, 1 GB。

n 名玩家使用一枚公平的 k 面骰子（骰子各面编号为 1 到 k ，即每次投掷得到 1 到 k 中任意点数的概率均为 $\frac{1}{k}$ ）进行骰子游戏。初始时，每位玩家的得分均为零。

在单次行动中，得分最少的玩家投掷骰子，并将投掷结果加到自己的得分上。若在某一时刻有多名玩家并列得分最低，则由其中随机一名玩家行动。

当任意一名玩家累计得分达到 m 分或以上时，游戏结束。求行动次数的期望值对 $10^9 + 7$ 取模。

$1 \leq n, k, m \leq 10^6$ 。2 s, 1 GB。

Hint 1: 当出现多位玩家并列得分最低时，可以强行钦定一个顺序，比如让编号小的玩家先行动。

n 名玩家使用一枚公平的 k 面骰子（骰子各面编号为 1 到 k ，即每次投掷得到 1 到 k 中任意点数的概率均为 $\frac{1}{k}$ ）进行骰子游戏。初始时，每位玩家的得分均为零。

在单次行动中，得分最少的玩家投掷骰子，并将投掷结果加到自己的得分上。若在某一时刻有多名玩家并列得分最低，则由其中随机一名玩家行动。

当任意一名玩家累计得分达到 m 分或以上时，游戏结束。求行动次数的期望值对 $10^9 + 7$ 取模。

$1 \leq n, k, m \leq 10^6$ 。2 s, 1 GB。

Hint 1：当出现多位玩家并列得分最低时，可以强行钦定一个顺序，比如让编号小的玩家先行动。

Hint 2：考虑去计算行动前得分最小值在每个值上的期望步数。

定义 f_i 为某一玩家经过得分 i 的概率, 定义 g_i 为行动前得分最小值在 i 上的期望步数。

先前缀和优化 $O(m)$ 求出 f_i 。

定义 f_i 为某一玩家经过得分 i 的概率, 定义 g_i 为行动前得分最小值在 i 上的期望步数。

先前缀和优化 $O(m)$ 求出 f_i 。

当 $i + k < m$ 时: 每个经过得分 i 的玩家都会对 g_i 产生 1 的贡献, 所以 $g_i = n f_i$ 。

定义 f_i 为某一玩家经过得分 i 的概率，定义 g_i 为行动前得分最小值在 i 上的期望步数。

先前缀和优化 $O(m)$ 求出 f_i 。

当 $i+k < m$ 时：每个经过得分 i 的玩家都会对 g_i 产生 1 的贡献，所以 $g_i = n f_i$ 。

当 $i+k \geq m$ 时：我们枚举经过得分 i 的玩家数量 j ，枚举经过得分 i 且行动前得分大于 i 的玩家数量 s 。

定义 h_i 为某一玩家不经过得分 i 且得分第一次超过 i 时的得分小于 m 的概率，令 $t = \frac{m-1-i}{k}$ ，则

$$g_i = \sum_{j=0}^n \binom{n}{j} f_i^j h_i^{n-j} \sum_{s=0}^{j-1} t^s = \frac{(f_i + h_i)^n - (t f_i + h_i)^n}{1 - t}。$$

定义 f_i 为某一玩家经过得分 i 的概率，定义 g_i 为行动前得分最小值在 i 上的期望步数。

先前缀和优化 $O(m)$ 求出 f_i 。

当 $i+k < m$ 时：每个经过得分 i 的玩家都会对 g_i 产生 1 的贡献，所以 $g_i = n f_i$ 。

当 $i+k \geq m$ 时：我们枚举经过得分 i 的玩家数量 j ，枚举经过得分 i 且行动前得分大于 i 的玩家数量 s 。

定义 h_i 为某一玩家不经过得分 i 且得分第一次超过 i 时的得分小于 m 的概率，令 $t = \frac{m-1-i}{k}$ ，则

$$g_i = \sum_{j=0}^n \binom{n}{j} f_i^j h_i^{n-j} \sum_{s=0}^{j-1} t^s = \frac{(f_i + h_i)^n - (t f_i + h_i)^n}{1 - t}。$$

而 h_i 也是容易求的，一种容易实现的方法是，直接用某一玩家经过大于 i 小于 m 的得分的概率和，减去从得分不小于 i 小于 m 操作到大于 i 小于 m 的概率和。

[QOJ 8702] 狼人杀

可怜和她的小伙伴们开了一局 n 名玩家参与的狼人杀。在这局游戏中，玩家从 1 到 n 编号，而可怜的编号是 m 。

与传统狼人杀不同的是，这局游戏包含一名狼人，一名预言家，和 $n-2$ 名村民。每晚只有预言家能够行动（即狼人并不能刀人）：预言家需要选择一个区间 $[l, r]$ ，而他会得知狼人的编号是否在这个区间中。

可怜分配到的身份是狼人，假设可怜的每名小伙伴都有 $\frac{1}{n-1}$ 的概率扮演预言家，且每晚预言家从所有区间中均匀选择（每个区间 $\frac{2}{n(n+1)}$ 的概率）。可怜想要知道期望在多少个夜晚后，预言家得到的信息能唯一确定可怜就是狼人。答案对 $10^9 + 7$ 取模。

$2 \leq n \leq 150, 1 \leq m \leq n$ 。8s, 256 MB，代码长度限制 8 KB。

[QOJ 8702] 狼人杀

可怜和她的小伙伴们开了一局 n 名玩家参与的狼人杀。在这局游戏中，玩家从 1 到 n 编号，而可怜的编号是 m 。

与传统狼人杀不同的是，这局游戏包含一名狼人，一名预言家，和 $n-2$ 名村民。每晚只有预言家能够行动（即狼人并不能刀人）：预言家需要选择一个区间 $[l, r]$ ，而他会得知狼人的编号是否在这个区间中。

可怜分配到的身份是狼人，假设可怜的每名小伙伴都有 $\frac{1}{n-1}$ 的概率扮演预言家，且每晚预言家从所有区间中均匀选择（每个区间 $\frac{2}{n(n+1)}$ 的概率）。可怜想要知道期望在多少个夜晚后，预言家得到的信息能唯一确定可怜就是狼人。答案对 $10^9 + 7$ 取模。

$2 \leq n \leq 150, 1 \leq m \leq n$ 。8s, 256 MB，代码长度限制 8 KB。

Hint 1: 每次询问都将所有玩家分成了有狼人和没有狼人的两个集合，能唯一确定可怜就是狼人的充要条件是所有村民都被某次询问中没有狼人的集合包含。

[QOJ 8702] 狼人杀

可怜和她的小伙伴们开了一局 n 名玩家参与的狼人杀。在这局游戏中，玩家从 1 到 n 编号，而可怜的编号是 m 。

与传统狼人杀不同的是，这局游戏包含一名狼人，一名预言家，和 $n-2$ 名村民。每晚只有预言家能够行动（即狼人并不能刀人）：预言家需要选择一个区间 $[l, r]$ ，而他会得知狼人的编号是否在这个区间中。

可怜分配到的身份是狼人，假设可怜的每名小伙伴都有 $\frac{1}{n-1}$ 的概率扮演预言家，且每晚预言家从所有区间中均匀选择（每个区间 $\frac{2}{n(n+1)}$ 的概率）。可怜想要知道期望在多少个夜晚后，预言家得到的信息能唯一确定可怜就是狼人。答案对 $10^9 + 7$ 取模。

$2 \leq n \leq 150, 1 \leq m \leq n$ 。8s, 256 MB，代码长度限制 8 KB。

Hint 1：每次询问都将所有玩家分成了有狼人和没有狼人的两个集合，能唯一确定可怜就是狼人的充要条件是所有村民都被某次询问中没有狼人的集合包含。

Hint 2：考虑 min-max 容斥。

把玩家放到一个环上，区间补是环上的区间。没有狼人的集合一定不包含 m ，我们再把环从 m 处断开并把 m 删除。

把玩家放到一个环上，区间补是环上的区间。没有狼人的集合一定不包含 m ，我们再把环从 m 处断开并把 m 删除。

答案形如对于所有非空村民集合 S ， $(-1)^{|S|-1}$ 乘上第一次询问中没有狼人的集合包含 $|S|$ 中的某个村民的期望步数，然后求和。

把玩家放到一个环上，区间补是环上的区间。没有狼人的集合一定不包含 m ，我们再把环从 m 处断开并把 m 删除。

答案形如对于所有非空村民集合 S ， $(-1)^{|S|-1}$ 乘上第一次询问中没有狼人的集合包含 $|S|$ 中的某个村民的期望步数，然后求和。

而对于每个 S ，后面那个乘上的东西只和合法的询问数有关，数这个直接记 S 中上一个在哪，上一个之前已经有的合法询问数，预言家是否已经出现，枚举这个在哪，转移是容易的。

交互库有一张 N 个点 M 条边的有标号简单无向连通图，满足每个点度数不超过 7，你已知 N ，要在不超过 45000 次询问后得到边集。

询问形如：你给出两个不同的点 s, t 和一个非空点集 S ，需满足 $s, t \in S$ ，交互库会告诉你在只能经过 S 中点的情况下， s, t 是否可达。

$2 \leq N \leq 1400, 1 \leq M \leq 1500$ 。2 s, 256 MB。

不清楚交互库使用的时间、空间，也不清楚是否自适应。

交互库有一张 N 个点 M 条边的有标号简单无向连通图，满足每个点度数不超过 7，你已知 N ，要在不超过 45000 次询问后得到边集。

询问形如：你给出两个不同的点 s, t 和一个非空点集 S ，需满足 $s, t \in S$ ，交互库会告诉你在只能经过 S 中点的情况下， s, t 是否可达。

$2 \leq N \leq 1400, 1 \leq M \leq 1500$ 。2s, 256 MB。

不清楚交互库使用的时间、空间，也不清楚是否自适应。

Hint 1：考虑以 0 和 $N-1$ 为端点的链怎么做。

交互库有一张 N 个点 M 条边的有标号简单无向连通图，满足每个点度数不超过 7，你已知 N ，要在不超过 45000 次询问后得到边集。

询问形如：你给出两个不同的点 s, t 和一个非空点集 S ，需满足 $s, t \in S$ ，交互库会告诉你在只能经过 S 中点的情况下， s, t 是否可达。

$2 \leq N \leq 1400, 1 \leq M \leq 1500$ 。2s, 256 MB。

不清楚交互库使用的时间、空间，也不清楚是否自适应。

Hint 1：考虑以 0 和 $N-1$ 为端点的链怎么做。

Hint 2：存在链的做法可以扩展到树。

交互库有一张 N 个点 M 条边的有标号简单无向连通图，满足每个点度数不超过 7，你已知 N ，要在不超过 45000 次询问后得到边集。

询问形如：你给出两个不同的点 s, t 和一个非空点集 S ，需满足 $s, t \in S$ ，交互库会告诉你在只能经过 S 中点的情况下， s, t 是否可达。

$2 \leq N \leq 1400, 1 \leq M \leq 1500$ 。2s, 256 MB。

不清楚交互库使用的时间、空间，也不清楚是否自适应。

Hint 1：考虑以 0 和 $N-1$ 为端点的链怎么做。

Hint 2：存在链的做法可以扩展到树。

Hint 3：想想树的做法还能否扩展。

先考虑以一种独特的方式解决以 0 和 $N - 1$ 为端点的链的情况：

先考虑以一种独特的方式解决以 0 和 $N - 1$ 为端点的链的情况：
维护一条当前已经确定下来的以 0 开头的链，记已被维护的点集为 S ，初始只有 0 一个点。
找一个目前不在 S 中的点 x ，考虑添加其及其到 S 的路径。

先考虑以一种独特的方式解决以 0 和 $N-1$ 为端点的链的情况：
维护一条当前已经确定下来的以 0 开头的链，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，考虑添加其及其到 S 的路径。

若 x 直接与 S 连通，则直接将 x 接在链的末尾；否则，需要找出 x 到 S 的路径后，将整条路径都加入。

先考虑以一种独特的方式解决以 0 和 $N-1$ 为端点的链的情况：
维护一条当前已经确定下来的以 0 开头的链，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，考虑添加其及其到 S 的路径。

若 x 直接与 S 连通，则直接将 x 接在链的末尾；否则，需要找出 x 到 S 的路径后，将整条路径都加入。

二分出 x 到 S 的路径上（不包括 x 和 S 中的点）编号最大的点 y ，将这条路径分成两半，递归继续找 y 到 S 的路径和 x 到 y 的路径。

先考虑以一种独特的方式解决以 0 和 $N-1$ 为端点的链的情况：
维护一条当前已经确定下来的以 0 开头的链，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，考虑添加其及其到 S 的路径。

若 x 直接与 S 连通，则直接将 x 接在链的末尾；否则，需要找出 x 到 S 的路径后，将整条路径都加入。

二分出 x 到 S 的路径上（不包括 x 和 S 中的点）编号最大的点 y ，将这条路径分成两半，递归继续找 y 到 S 的路径和 x 到 y 的路径。

这样做需要不超过 $n \lceil \log_2 n \rceil + n$ 次询问，虽然不是最优的方法，但其具有可扩展性。

树的情况：

树的情况：

与链类似的，维护一个包含 0 的连通块，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其及其到 S 的路径。

树的情况：

与链类似的，维护一个包含 0 的连通块，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其及其到 S 的路径。

仍然采用找编号最大点并递归做的方法，可以找到路径上除了最后一条边以外其它边。与链不同的是，还要找到路径上直接与 S 中点相连的边连向了 S 中哪个点。

树的情况：

与链类似的，维护一个包含 0 的连通块，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其及其到 S 的路径。

仍然采用找编号最大点并递归做的方法，可以找到路径上除了最后一条边以外其它边。与链不同的是，还要找到路径上直接与 S 中点相连的边连向了 S 中哪个点。

考虑从 0 开始在 S 内进行 dfs，并记录下每个点的 dfn。

记路径上不在 S 中的最后一个点为 z ，二分出最短的 dfn 前缀使 z 与其连通，并记这个前缀的最后一个点为 u 。

树的情况：

与链类似的，维护一个包含 0 的连通块，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其及其到 S 的路径。

仍然采用找编号最大点并递归做的方法，可以找到路径上除了最后一条边以外其它边。与链不同的是，还要找到路径上直接与 S 中点相连的边连向了 S 中哪个点。

考虑从 0 开始在 S 内进行 dfs，并记录下每个点的 dfn。

记路径上不在 S 中的最后一个点为 z ，二分出最短的 dfn 前缀使 z 与其连通，并记这个前缀的最后一个点为 u 。

则可以证明 u 和 z 间有直接连边。

树的情况：

与链类似的，维护一个包含 0 的连通块，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其及其到 S 的路径。

仍然采用找编号最大点并递归做的方法，可以找到路径上除了最后一条边以外其它边。与链不同的是，还要找到路径上直接与 S 中点相连的边连向了 S 中哪个点。

考虑从 0 开始在 S 内进行 dfs，并记录下每个点的 dfn。

记路径上不在 S 中的最后一个点为 z ，二分出最短的 dfn 前缀使 z 与其连通，并记这个前缀的最后一个点为 u 。

则可以证明 u 和 z 间有直接连边。

仍然只需不超过 $n \lceil \log_2 n \rceil + n$ 次询问。

图的情况：

图的情况：

同样维护一个包含 0 的连通块，找出连通块内部的所有边，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其到 S 的某一条路径上所有不在 S 中的点。

图的情况：

同样维护一个包含 0 的连通块，找出连通块内部的所有边，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其到 S 的某一条路径上所有不在 S 中的点。

仍然套用二分找编号最大点并递归做的方法，找到了一条路径，记路径上不在 S 中的最后一个点为 z ，发现路径上除 z 以外不在 S 中的点都没有其它与 S 中或路径上的点相连的边。

图的情况：

同样维护一个包含 0 的连通块，找出连通块内部的所有边，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其到 S 的某一条路径上所有不在 S 中的点。

仍然套用二分找编号最大点并递归做的方法，找到了一条路径，记路径上不在 S 中的最后一个点为 z ，发现路径上除 z 以外不在 S 中的点都没有其它与 S 中或路径上的点相连的边。

在 dfn 上仍然可以找到一条 u, z 间的边。

图的情况：

同样维护一个包含 0 的连通块，找出连通块内部的所有边，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其到 S 的某一条路径上所有不在 S 中的点。

仍然套用二分找编号最大点并递归做的方法，找到了一条路径，记路径上不在 S 中的最后一个点为 z ，发现路径上除 z 以外不在 S 中的点都没有其它与 S 中或路径上的点相连的边。

在 dfn 上仍然可以找到一条 u, z 间的边。

类似点分治，找出 S 删掉 u 后裂成的不超过 6 个连通块。

先花费一次询问问一下 z 是否与该连通块有直接连边，若有再继续递归。

图的情况：

同样维护一个包含 0 的连通块，找出连通块内部的所有边，记已被维护的点集为 S ，初始只有 0 一个点。

找一个目前不在 S 中的点 x ，添加其到 S 的某一条路径上所有不在 S 中的点。

仍然套用二分找编号最大点并递归做的方法，找到了一条路径，记路径上不在 S 中的最后一个点为 z ，发现路径上除 z 以外不在 S 中的点都没有其它与 S 中或路径上的点相连的边。

在 dfn 上仍然可以找到一条 u, z 间的边。

类似点分治，找出 S 删掉 u 后裂成的不超过 6 个连通块。

先花费一次询问问一下 z 是否与该连通块有直接连边，若有再继续递归。

这样做，每条边只需均摊掉 6 次多余询问。

因此最后询问总数不超过 $m \lceil \log_2 n \rceil + n + 6m \leq 26900$ ，这远小于 45000 。

给定两个 $N \times N$ 的 01 矩阵 A, B , 构造一种执行不超过 10^3 次以下操作的方案使矩阵从 A 变成 B :

从 $[1, N]$ 中选择一个整数 i , 将第 i 行向左循环位移一位;

从 $[1, N]$ 中选择一个整数 i , 将第 i 列向下循环位移一位后, 将第 1 行第 i 列的值异或 1。

$3 \leq N \leq 20$ 。1 s, 1 GB。

给定两个 $N \times N$ 的 01 矩阵 A, B , 构造一种执行不超过 10^3 次以下操作的方案使矩阵从 A 变成 B :

从 $[1, N]$ 中选择一个整数 i , 将第 i 行向左循环位移一位;

从 $[1, N]$ 中选择一个整数 i , 将第 i 列向下循环位移一位后, 将第 1 行第 i 列的值异或 1。

$3 \leq N \leq 20$ 。1 s, 1 GB。

Hint: 考虑一行行或一列列地修改每个位置上的值。

由于我提出了一个完全不同于题解的做法，在此先来看题解是怎么做的。

由于我提出了一个完全不同于题解的做法，在此先来看题解是怎么做的。

首先考虑下面过程：

从左往右枚举每一列 $c (1 \leq c < n)$ ，先对 $a_{n,c} = b_{i,c}$ 的 i 执行 row i ，然后执行 n 次 column n ，再把刚才没操作的行 i 执行 row i 。容易发现，对于一个 c 执行完上述操作后， a 的第 $n-1$ 列会与 b 的第 c 列相同。

执行完后，最后一列可能不满足要求。

由于我提出了一个完全不同于题解的做法，在此先来看题解是怎么做的。

首先考虑下面过程：

从左往右枚举每一列 $c(1 \leq c < n)$ ，先对 $a_{n,c} = b_{i,c}$ 的 i 执行 row i ，然后执行 n 次 column n ，再把刚才没操作的行 i 执行 row i 。容易发现，对于一个 c 执行完上述操作后， a 的第 $n-1$ 列会与 b 的第 c 列相同。

执行完后，最后一列可能不满足要求。

观察到执行完上面操作后， a 每行 1 的个数的奇偶性的变化是固定的：row 操作不会改变行奇偶性，连续 n 次 column n 会同时改变每行的奇偶性。所以如果能够先处理 a 的行奇偶性和 b 的行奇偶性完全相同/相反（取决于 n 的奇偶性）即可。

这样就可以先计算出每一行需要将 $a_{i,n}$ 调整为多少，记这个值为 A_i 。

现在只要调整奇偶性。先把第 n 列和第 n 行变成全 0，然后再把第 n 行 $\text{column } n$ 次变成全 1。由于将所有 01 反转实际上没有区别，下面假设第 n 行和第 n 列的 $2n - 1$ 个数中 1 比 0 多。

考虑将第 n 行和第 n 列先全部变成全 0，然后再由全 0 还原出目标奇偶性。

现在只要调整奇偶性。先把第 n 列和第 n 行变成全 0，然后再把第 n 行 $\text{column } n$ 次变成全 1。由于将所有 01 反转实际上没有区别，下面假设第 n 行和第 n 列的 $2n-1$ 个数中 1 比 0 多。

考虑将第 n 行和第 n 列先全部变成全 0，然后再由全 0 还原出目标奇偶性。

要将第 n 行和第 n 列先全部变成全 0，可以先把第 n 列清零。这一步的做法是执行 n 次“不断执行 $\text{row } n$ 直到 $a_{n,n} = 1$ ，然后 $\text{column } n$ ”。这样会把 n 个 1 变成 0 并放到第 n 列。

接下来把第 n 行清零。这一步只要不断把第 n 行的 1 用 $\text{row } n$ 放到 (n, n) 的位置，然后 $\text{column } n$ 。

完成到这一步之后，column n 次，把第 n 列变为全 1。此时 a 会长下面这个样子

?	?	...	?	1
?	?	...	?	1
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
?	?	...	?	1
0	0	...	0	1

完成到这一步之后，column n n 次，把第 n 列变为全 1。此时 a 会长下面这个样子

$$\begin{array}{ccccc}
 ? & ? & \cdots & ? & 1 \\
 ? & ? & \cdots & ? & 1 \\
 \vdots & \vdots & \ddots & \vdots & \vdots \\
 ? & ? & \cdots & ? & 1 \\
 0 & 0 & \cdots & 0 & 1
 \end{array}$$

先忽略掉 A_n ，解决掉 $A_1 \cdots A_{n-1}$ 。类似最开始说的东西，依次考虑 $A_{n-1} \cdots A_1$ ，如果是一个 1 就从第 n 行拉一个 0 过来然后 column n ，否则直接 column n 。对于第 n 行的奇偶性，调整是简单的。

完成到这一步之后，column n 次，把第 n 列变为全 1。此时 a 会长下面这个样子

$$\begin{array}{ccccc} ? & ? & \cdots & ? & 1 \\ ? & ? & \cdots & ? & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ ? & ? & \cdots & ? & 1 \\ 0 & 0 & \cdots & 0 & 1 \end{array}$$

先忽略掉 A_n ，解决掉 $A_1 \cdots A_{n-1}$ 。类似最开始说的东西，依次考虑 $A_{n-1} \cdots A_1$ ，如果是一个 1 就从第 n 行拉一个 0 过来然后 column n ，否则直接 column n 。对于第 n 行的奇偶性，调整是简单的。

前面部分的操作次数是 $2n(n-1)$ ，调整奇偶性的次数是 $O(n)$ 的。可以通过。

再简单介绍一下我的做法。

再简单介绍一下我的做法。

用 $4n$ 次操作，将 A 的第 1 列变成全 1 第 2 列变成全 0，同理再用 $4n$ 次逆操作将 B 的第 2 列变成全 1 第 1 列变成全 0。

再简单介绍一下我的做法。

用 $4n$ 次操作，将 A 的第 1 列变成全 1 第 2 列变成全 0，同理再用 $4n$ 次逆操作将 B 的第 2 列变成全 1 第 1 列变成全 0。

接着，从上到下处理每一行。此时，这行第 1、2 个位置成了一个 $1 \rightarrow 0, 0 \rightarrow 1$ 的转换器。进行 $n - 2$ 次 row 操作，并合理使用转换器。总次数 $2n^2 + 6n$ 。

再简单介绍一下我的做法。

用 $4n$ 次操作，将 A 的第 1 列变成全 1 第 2 列变成全 0，同理再用 $4n$ 次逆操作将 B 的第 2 列变成全 1 第 1 列变成全 0。

接着，从上到下处理每一行。此时，这行第 1、2 个位置成了一个 $1 \rightarrow 0, 0 \rightarrow 1$ 的转换器。进行 $n-2$ 次 row 操作，并合理使用转换器。总次数 $2n^2 + 6n$ 。

而你再套个分块上去，先把 A 变成每 t 列有一列全 1 和一系列全 0， B 变成每 t 列有一列全 0 一系列全 1，则你可以对每行只进行 $t-2$ 次 row 操作。总次数 $n^2 - 2n + nt + 8n\lceil \frac{n}{t} \rceil$ ，即可以做到不超过 $n^2 + 4\sqrt{2n}\sqrt{n} + 7n$ 次。

再简单介绍一下我的做法。

用 $4n$ 次操作，将 A 的第 1 列变成全 1 第 2 列变成全 0，同理再用 $4n$ 次逆操作将 B 的第 2 列变成全 1 第 1 列变成全 0。

接着，从上到下处理每一行。此时，这行第 1、2 个位置成了一个 $1 \rightarrow 0, 0 \rightarrow 1$ 的转换器。进行 $n-2$ 次 row 操作，并合理使用转换器。总次数 $2n^2 + 6n$ 。

而你再套个分块上去，先把 A 变成每 t 列有一列全 1 和一系列全 0， B 变成每 t 列有一列全 0 一系列全 1，则你可以对每行只进行 $t-2$ 次 row 操作。总次数 $n^2 - 2n + nt + 8n\lceil \frac{n}{t} \rceil$ ，即可以做到不超过 $n^2 + 4\sqrt{2}n\sqrt{n} + 7n$ 次。

事实上，可以继续优化。在 $n \leq 10^3$ 的情况下，次数比较优秀的大致可以做到 $n^2 + 4n\sqrt{n} + O(n)$ 。其它一些再套一次分块的做法，的确可以减小 $n\sqrt{n}$ 前的系数到不超过 $2\sqrt{2}$ ，但会多出若干倍的 $n^{1.25}$ ，应该不太优秀。

交互库有一个元素为 $[0, 2^b)$ 中的整数的数组 $a_1, a_2, \dots, a_n$ ，你要回答其中的最大值，你可以进行不超过 $3(n + b)$ 次询问，每次询问由你先给出两个数 i, y ，交互库会告诉你 $[a_i > y]$ 。交互库自适应。

$1 \leq n, b \leq 200$ 。1 s, 256 MB。

不清楚交互库使用的时间。

交互库有一个元素为 $[0, 2^b)$ 中的整数的数组 $a_1, a_2, \dots, a_n$ ，你要回答其中的最大值，你可以进行不超过 $3(n + b)$ 次询问，每次询问由你先给出两个数 i, y ，交互库会告诉你 $[a_i > y]$ 。交互库自适应。

$1 \leq n, b \leq 200$ 。1 s, 256 MB。

不清楚交互库使用的时间。

Hint：考虑花费若干次询问递归到一个子问题。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

从前往后扫序列，维护一个形如 $\overline{s_1 \dots}, \overline{s_1 s_2 \dots}, \dots, \overline{s_1 s_2 \dots s_k \dots}$ 的子序列，记其中第 i 个数在原序列中的位置为 c_i ，则还应满足对于所有 i ，原序列在第 c_i 个数之后中的数的前 i 位不大于 a_{c_i} 的前 i 位。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

从前往后扫序列，维护一个形如 $\overline{s_1 \dots}, \overline{s_1 s_2 \dots}, \dots, \overline{s_1 s_2 \dots s_k \dots}$ 的子序列，记其中第 i 个数在原序列中的位置为 c_i ，则还应满足对于所有 i ，原序列在第 c_i 个数之后中的数的前 i 位不大于 a_{c_i} 的前 i 位。

具体地，考虑扫到第 j 个数时，执行以下过程：

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

从前往后扫序列，维护一个形如 $\overline{s_1 \dots}, \overline{s_1 s_2 \dots}, \dots, \overline{s_1 s_2 \dots s_k \dots}$ 的子序列，记其中第 i 个数在原序列中的位置为 c_i ，则还应满足对于所有 i ，原序列在第 c_i 个数之后中的数的前 i 位不大于 a_{c_i} 的前 i 位。

具体地，考虑扫到第 j 个数时，执行以下过程：

1. 若 a_j 的前 k 位小于 a_{c_k} 的前 k 位，则直接扫下一个数。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

从前往后扫序列，维护一个形如 $\overline{s_1 \dots}, \overline{s_1 s_2 \dots}, \dots, \overline{s_1 s_2 \dots s_k \dots}$ 的子序列，记其中第 i 个数在原序列中的位置为 c_i ，则还应满足对于所有 i ，原序列在第 c_i 个数之后中的数的前 i 位不大于 a_{c_i} 的前 i 位。

具体地，考虑扫到第 j 个数时，执行以下过程：

1. 若 a_j 的前 k 位小于 a_{c_k} 的前 k 位，则直接扫下一个数。
2. 若 a_j 的前 k 位大于 a_{c_k} 的前 k 位，则删除子序列的最后一项，此时 k 减小了一，重新判断 a_j 的前 k 位和 a_{c_k} 的前 k 位的大小关系，如果还是大于就继续删除，直至出现等于。

考虑花费若干次询问，缩小最大值可能出现的位置集合 S ，同时确定最大值的最高若干位。

首先试着提出一种关于 n, b 线性的做法。

从前往后扫序列，维护一个形如 $\overline{s_1 \dots}, \overline{s_1 s_2 \dots}, \dots, \overline{s_1 s_2 \dots s_k \dots}$ 的子序列，记其中第 i 个数在原序列中的位置为 c_i ，则还应满足对于所有 i ，原序列在第 c_i 个数之后中的数的前 i 位不大于 a_{c_i} 的前 i 位。

具体地，考虑扫到第 j 个数时，执行以下过程：

1. 若 a_j 的前 k 位小于 a_{c_k} 的前 k 位，则直接扫下一个数。
2. 若 a_j 的前 k 位大于 a_{c_k} 的前 k 位，则删除子序列的最后一项，此时 k 减小了一，重新判断 a_j 的前 k 位和 a_{c_k} 的前 k 位的大小关系，如果还是大于就继续删除，直至出现等于。
3. 确定 a_j 的第 $k+1$ 位，并将其加入子序列的末尾。

扫完整个序列之后，得到一个子序列，考虑再从前往后扫子序列中的数，每次扫到一个数，执行与之前几乎一致的过程，但这次我们不执行步骤 3.（其实等价于只执行步骤 2.），扫到某个现在已经被删除了的数就结束。

扫完整个序列之后，得到一个子序列，考虑再从前往后扫子序列中的数，每次扫到一个数，执行与之前几乎一致的过程，但这次我们不执行步骤 3.（其实等价于只执行步骤 2.），扫到某个现在已经被删除了的数就结束。

这样做完之后，我们直接发现最大值的前 k 位就等于 a_{c_k} 的前 k 位，而且最大值只可能出现在最终的子序列中。我们就成功递归子问题了，但分析后发现询问次数为 $4(n + b)$ 。

扫完整个序列之后，得到一个子序列，考虑再从前往后扫子序列中的数，每次扫到一个数，执行与之前几乎一致的过程，但这次我们不执行步骤 3. (其实等价于只执行步骤 2.)，扫到某个现在已经被删除了的数就结束。

这样做完之后，我们直接发现最大值的前 k 位就等于 a_{c_k} 的前 k 位，而且最大值只可能出现在最终的子序列中。我们就成功递归子问题了，但分析后发现询问次数为 $4(n + b)$ 。

直接扔掉步骤 1.，会发生什么？假如一个数本应在步骤 1. 被跳过，现在你会在确定第 $k + 1$ 位时认为那位是 0。

扫完整个序列之后，得到一个子序列，考虑再从前往后扫子序列中的数，每次扫到一个数，执行与之前几乎一致的过程，但这次我们不执行步骤 3. (其实等价于只执行步骤 2.)，扫到某个现在已经被删除了的数就结束。

这样做完之后，我们直接发现最大值的前 k 位就等于 a_{c_k} 的前 k 位，而且最大值只可能出现在最终的子序列中。我们就成功递归子问题了，但分析后发现询问次数为 $4(n + b)$ 。

直接扔掉步骤 1.，会发生什么？假如一个数本应在步骤 1. 被跳过，现在你会在确定第 $k + 1$ 位时认为那位是 0。

因此你所认为的值不会小于实际的值，且可以说明任意时刻都至少有一个子序列中的数的前 k 位不小于 $\overline{s_1 s_2 \dots s_k}$ 。

扫完整个序列之后，得到一个子序列，考虑再从前往后扫子序列中的数，每次扫到一个数，执行与之前几乎一致的过程，但这次我们不执行步骤 3. (其实等价于只执行步骤 2.)，扫到某个现在已经被删除了的数就结束。

这样做完之后，我们直接发现最大值的前 k 位就等于 a_{c_k} 的前 k 位，而且最大值只可能出现在最终的子序列中。我们就成功递归子问题了，但分析后发现询问次数为 $4(n + b)$ 。

直接扔掉步骤 1.，会发生什么？假如一个数本应在步骤 1. 被跳过，现在你会在确定第 $k + 1$ 位时认为那位是 0。

因此你所认为的值不会小于实际的值，且可以说明任意时刻都至少有一个子序列中的数的前 k 位不小于 $\overline{s_1 s_2 \dots s_k}$ 。

扫完原序列，再扫完子序列后，你仍然能获得正确的最大值前 k 位，和最大值可能出现集合，而询问次数变成了 $3(n + b)$ 。

昨天，我室友说这其实是可以 $3n + 2b$ 的。

具体地，你发现根本没有必要去递归下去。你扫完序列之后，扫子序列时，仍然执行步骤 3.，并一直扫下去。

昨天，我室友说这其实是可以 $3n + 2b$ 的。

具体地，你发现根本没有必要去递归下去。你扫完序列之后，扫子序列时，仍然执行步骤 3.，并一直扫下去。

分析询问次数时，你可以把每个数最后一次加入摊到 n 上，其余都摊到 b 上，直接就是 $3n + 2b$ 了。

昨天，我室友说这其实是可以 $3n + 2b$ 的。

具体地，你发现根本没有必要去递归下去。你扫完序列之后，扫子序列时，仍然执行步骤 3.，并一直扫下去。

分析询问次数时，你可以把每个数最后一次加入摊到 n 上，其余都摊到 b 上，直接就是 $3n + 2b$ 了。

经过他的启发，我发现 $2n + 3b$ 也是容易做到的。

昨天，我室友说这其实是可以 $3n + 2b$ 的。

具体地，你发现根本没有必要去递归下去。你扫完序列之后，扫子序列时，仍然执行步骤 3.，并一直扫下去。

分析询问次数时，你可以把每个数最后一次加入摊到 n 上，其余都摊到 b 上，直接就是 $3n + 2b$ 了。

经过他的启发，我发现 $2n + 3b$ 也是容易做到的。

那这两个一拼就 $\frac{5}{2}(n + b)$ 了。

牌堆有 n 张牌，从顶到底第 s 张为关键牌，关键牌初始分值为 1。小 A 和小 B 都知道关键牌的位置。小 A 和小 B 轮流操作，小 A 先手。每次行动时，当前行动人可做如下操作：

1. 取出牌堆顶的 k 张牌， k 为一给定常数。若牌堆牌数不足 k 则取全部。然后选择移除其中的 0 或 1 张牌，并将其它取出牌以自己指定的顺序放在牌堆底。
2. 若小 B 移除了关键牌，则游戏立刻结束，得分为 0。
3. 若小 A 移除了关键牌，则游戏立刻结束，得分为关键牌当前分值。
4. 若小 A 取出了关键牌并将它放回牌堆底，则他要告诉小 B 新的关键牌位置，且关键牌的分值变成原来加 1。

小 A 希望得分尽可能大，小 B 希望得分尽可能小。现有 T 组数据，问如果双方都以最优策略行动，最后的得分。

$1 \leq T \leq 50, 1 \leq n, k \leq 10^9, 1 \leq s \leq n$ 。3s, 512 MB。

牌堆有 n 张牌，从顶到底第 s 张为关键牌，关键牌初始分值为 1。小 A 和小 B 都知道关键牌的位置。小 A 和小 B 轮流操作，小 A 先手。每次行动时，当前行动人可做如下操作：

1. 取出牌堆顶的 k 张牌， k 为一给定常数。若牌堆牌数不足 k 则取全部。然后选择移除其中的 0 或 1 张牌，并将其它取出牌以自己指定的顺序放在牌堆底。
2. 若小 B 移除了关键牌，则游戏立刻结束，得分为 0。
3. 若小 A 移除了关键牌，则游戏立刻结束，得分为关键牌当前分值。
4. 若小 A 取出了关键牌并将它放回牌堆底，则他要告诉小 B 新的关键牌位置，且关键牌的分值变成原来加 1。

小 A 希望得分尽可能大，小 B 希望得分尽可能小。现有 T 组数据，问如果双方都以最优策略行动，最后的得分。

$1 \leq T \leq 50, 1 \leq n, k \leq 10^9, 1 \leq s \leq n$ 。3 s, 512 MB。

Hint 1: 考虑怎么 $O(\sum n)$ 做。

牌堆有 n 张牌，从顶到底第 s 张为关键牌，关键牌初始分值为 1。小 A 和小 B 都知道关键牌的位置。小 A 和小 B 轮流操作，小 A 先手。每次行动时，当前行动人可做如下操作：

1. 取出牌堆顶的 k 张牌， k 为一给定常数。若牌堆牌数不足 k 则取全部。然后选择移除其中的 0 或 1 张牌，并将其它取出牌以自己指定的顺序放在牌堆底。
2. 若小 B 移除了关键牌，则游戏立刻结束，得分为 0。
3. 若小 A 移除了关键牌，则游戏立刻结束，得分为关键牌当前分值。
4. 若小 A 取出了关键牌并将它放回牌堆底，则他要告诉小 B 新的关键牌位置，且关键牌的分值变成原来加 1。

小 A 希望得分尽可能大，小 B 希望得分尽可能小。现有 T 组数据，问如果双方都以最优策略行动，最后的得分。

$1 \leq T \leq 50, 1 \leq n, k \leq 10^9, 1 \leq s \leq n$ 。3 s, 512 MB。

Hint 1: 考虑怎么 $O(\sum n)$ 做。

Hint 2: 试试打表或手模找找性质。

考虑 $n \geq k, s \leq k$ 的情况, 若小 A 第一次行动无法把关键牌放到之后由自己选择的位置, 即 $n \equiv k \pmod{2k}$, 则小 A 只能在第一次行动时结束游戏, 获得 1 分。

考虑 $n \geq k, s \leq k$ 的情况, 若小 A 第一次行动无法把关键牌放到之后由自己选择的位置, 即 $n \equiv k \pmod{2k}$, 则小 A 只能在第一次行动时结束游戏, 获得 1 分。

否则小 A 会把关键牌放到之后由自己选择的位置, 接着双方各进行 $x = \lfloor \frac{n+k}{2k} \rfloor$ 次行动, 然后回到 $s \leq k$, 但此时剩余牌数不固定。

考虑 $n \geq k, s \leq k$ 的情况, 若小 A 第一次行动无法把关键牌放到之后由自己选择的位置, 即 $n \equiv k \pmod{2k}$, 则小 A 只能在第一次行动时结束游戏, 获得 1 分。

否则小 A 会把关键牌放到之后由自己选择的位置, 接着双方各进行 $x = \lfloor \frac{n+k}{2k} \rfloor$ 次行动, 然后回到 $s \leq k$, 但此时剩余牌数不固定。

我们先假定能求出所有可能剩余牌数的答案, 然后有了一步相当神的操作。

考虑一共有 $2x$ 次行动, 可能会移除 $0 \sim 2x$ 张牌。我们移动一下坐标系, 看成, 一开始最后会剩 $n - x$ 张牌。一个人移除牌就相当于减少 $\frac{1}{2}$ 张牌, 不移除就是增加 $\frac{1}{2}$ 张牌。

考虑 $n \geq k, s \leq k$ 的情况, 若小 A 第一次行动无法把关键牌放到之后由自己选择的位置, 即 $n \equiv k \pmod{2k}$, 则小 A 只能在第一次行动时结束游戏, 获得 1 分。

否则小 A 会把关键牌放到之后由自己选择的位置, 接着双方各进行 $x = \lfloor \frac{n+k}{2k} \rfloor$ 次行动, 然后回到 $s \leq k$, 但此时剩余牌数不固定。

我们先假定能求出所有可能剩余牌数的答案, 然后有了一步相当神的操作。

考虑一共有 $2x$ 次行动, 可能会移除 $0 \sim 2x$ 张牌。我们移动一下坐标系, 看成, 一开始最后会剩 $n - x$ 张牌。一个人移除牌就相当于减少 $\frac{1}{2}$ 张牌, 不移除就是增加 $\frac{1}{2}$ 张牌。

对小 B 来说, 小 A 先往左或往右走 $\frac{1}{2}$, 然后小 B 可从当前相邻的两个中选一个, 之后小 B 只要一直走和小 A 相反的就可以达到他能选的二者中较小的那个, 即最后得分小于等于这个。

考虑 $n \geq k, s \leq k$ 的情况, 若小 A 第一次行动无法把关键牌放到之后由自己选择的位置, 即 $n \equiv k \pmod{2k}$, 则小 A 只能在第一次行动时结束游戏, 获得 1 分。

否则小 A 会把关键牌放到之后由自己选择的位置, 接着双方各进行 $x = \lfloor \frac{n+k}{2k} \rfloor$ 次行动, 然后回到 $s \leq k$, 但此时剩余牌数不固定。

我们先假定能求出所有可能剩余牌数的答案, 然后有了一步相当神的操作。

考虑一共有 $2x$ 次行动, 可能会移除 $0 \sim 2x$ 张牌。我们移动一下坐标系, 看成, 一开始最后会剩 $n - x$ 张牌。一个人移除牌就相当于减少 $\frac{1}{2}$ 张牌, 不移除就是增加 $\frac{1}{2}$ 张牌。

对小 B 来说, 小 A 先往左或往右走 $\frac{1}{2}$, 然后小 B 可从当前相邻的两个中选一个, 之后小 B 只要一直走和小 A 相反的就可以达到他能选的二者中较小的那个, 即最后得分小于等于这个。

而对小 A 来说, 他除了第一次行动以外一直走和小 B 相反的也一定可以做到让得分大于等于这二者之间较小的那个。

考虑 $n \geq k, s \leq k$ 的情况，若小 A 第一次行动无法把关键牌放到之后由自己选择的位置，即 $n \equiv k \pmod{2k}$ ，则小 A 只能在第一次行动时结束游戏，获得 1 分。

否则小 A 会把关键牌放到之后由自己选择的位置，接着双方各进行 $x = \lfloor \frac{n+k}{2k} \rfloor$ 次行动，然后回到 $s \leq k$ ，但此时剩余牌数不固定。

我们先假定能求出所有可能剩余牌数的答案，然后有了一步相当神的操作。

考虑一共有 $2x$ 次行动，可能会移除 $0 \sim 2x$ 张牌。我们移动一下坐标系，看成，一开始最后会剩 $n - x$ 张牌。一个人移除牌就相当于减少 $\frac{1}{2}$ 张牌，不移除就是增加 $\frac{1}{2}$ 张牌。

对小 B 来说，小 A 先往左或往右走 $\frac{1}{2}$ ，然后小 B 可从当前相邻的两个中选一个，之后小 B 只要一直走和小 A 相反的就可以达到他能选的二者中较小的那个，即最后得分小于等于这个。

而对小 A 来说，他除了第一次行动以外一直走和小 B 相反的也一定可以做到让得分大于等于这二者之间较小的那个。

发现对 $s > k$ ，也可以用同样的道理，转化成 $s \leq k$ 。

特别地，若初始 $n \equiv k + 1 \pmod{2k}$ ，则小 A 第一次行动只能移除 0 张牌。

特别地，若初始 $n \equiv k + 1 \pmod{2k}$ ，则小 A 第一次行动只能移除 0 张牌。

因此，假如我们记 $f(n)$ 为剩余 n 张牌 $s \leq k$ 时的答案，令 $p = n - x, m = n \bmod 2k$ ，则

$$f(n) = \begin{cases} 1, & m = k \\ 1 + \min\{f(p), f(p + 1)\}, & m = k + 1 \\ 1 + \min\{\max\{f(p - 1), f(p + 1)\}, f(p)\}, & \text{otherwise} \end{cases}$$

这么表示在 $k = 1$ 时可能并不怎么正确，但我懒得改了，只要大家看得懂就行。

当然，最后一种等价于

$$1 + \max\{\min\{f(p - 1), f(p)\}, \min\{f(p), f(p + 1)\}\}.$$

n 较小时可能会出现自转移，直接认为调用的 $f(n) = +\infty$ 即可。

特别地，若初始 $n \equiv k + 1 \pmod{2k}$ ，则小 A 第一次行动只能移除 0 张牌。

因此，假如我们记 $f(n)$ 为剩余 n 张牌 $s \leq k$ 时的答案，令 $p = n - x, m = n \bmod 2k$ ，则

$$f(n) = \begin{cases} 1, & m = k \\ 1 + \min\{f(p), f(p + 1)\}, & m = k + 1 \\ 1 + \min\{\max\{f(p - 1), f(p + 1)\}, f(p)\}, & \text{otherwise} \end{cases}$$

这么表示在 $k = 1$ 时可能并不怎么正确，但我懒得改了，只要大家看得懂就行。

当然，最后一种等价于

$$1 + \max\{\min\{f(p - 1), f(p)\}, \min\{f(p), f(p + 1)\}\}.$$

n 较小时可能会出现自转移，直接认为调用的 $f(n) = +\infty$ 即可。那就可以 $O(n)$ 处理一次询问了。考虑进行打表或手模并观察。

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。
考虑去证明这件事。

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。

考虑去证明这件事。

除那两种情况外，不从其转移，即 $f(p) > \max\{f(p-1), f(p+1)\}$ 。

为了方便，我们重新定义自转移时的边界，在第一层调用 $f(n)$ 仍继续递归，认为第二层调用的 $f(n) = +\infty$ 。

记 $s = p \bmod 2k, t = p - \lfloor \frac{p+k}{2k} \rfloor$ 。

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。

考虑去证明这件事。

除那两种情况外，不从其转移，即 $f(p) > \max\{f(p-1), f(p+1)\}$ 。

为了方便，我们重新定义自转移时的边界，在第一层调用 $f(n)$ 仍继续递归，认为第二层调用的 $f(n) = +\infty$ 。

记 $s = p \bmod 2k, t = p - \lfloor \frac{p+k}{2k} \rfloor$ 。

- 若 $s = k$ ，则 $f(p) = 1 \leq \max\{f(p-1), f(p+1)\}$;

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。

考虑去证明这件事。

除那两种情况外，不从其转移，即 $f(p) > \max\{f(p-1), f(p+1)\}$ 。

为了方便，我们重新定义自转移时的边界，在第一层调用 $f(n)$ 仍继续递归，认为第二层调用的 $f(n) = +\infty$ 。

记 $s = p \bmod 2k, t = p - \lfloor \frac{p+k}{2k} \rfloor$ 。

- 若 $s = k$ ，则 $f(p) = 1 \leq \max\{f(p-1), f(p+1)\}$;
- 若 $s = k + 1$ ，则 $f(p) = 1 + \min\{f(t+1), f(t)\} \leq \max\{1, 1 + \min\{\max\{f(t-1), f(t+1)\}, f(t)\}\} = \max\{f(p-1), f(p+1)\}$;

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。

考虑去证明这件事。

除那两种情况外，不从其转移，即 $f(p) > \max\{f(p-1), f(p+1)\}$ 。

为了方便，我们重新定义自转移时的边界，在第一层调用 $f(n)$ 仍继续递归，认为第二层调用的 $f(n) = +\infty$ 。

记 $s = p \bmod 2k, t = p - \lfloor \frac{p+k}{2k} \rfloor$ 。

- 若 $s = k$ ，则 $f(p) = 1 \leq \max\{f(p-1), f(p+1)\}$;
- 若 $s = k + 1$ ，则 $f(p) = 1 + \min\{f(t+1), f(t)\} \leq \max\{1, 1 + \min\{\max\{f(t-1), f(t+1)\}, f(t)\}\} = \max\{f(p-1), f(p+1)\}$;
- 否则若 $s \neq k - 1$ ，则 $\lfloor \frac{p-1+k}{2k} \rfloor = \lfloor \frac{p+k}{2k} \rfloor = \lfloor \frac{p+1+k}{2k} \rfloor$ ，记 $a_1 = \min\{f(t-2), f(t-1)\}, a_2 = \min\{f(t-1), f(t)\}, a_3 = \min\{f(t), f(t+1)\}, a_4 = \min\{f(t+1), f(t+2)\}$ ，则 $f(p) = 1 + \max\{a_2, a_3\} \leq 1 + \max\{a_1, a_2, a_3, a_4\} = \max\{f(p-1), f(p+1)\}$ 。

发现大部分 $f(n)$ 都从 $1 + f(p)$ 转移而来，而不从其转移的，除 $m = k$ 和 $m = k + 1$ 外，都满足 $p \equiv k - 1 \pmod{2k}$ 。

考虑去证明这件事。

除那两种情况外，不从其转移，即 $f(p) > \max\{f(p-1), f(p+1)\}$ 。

为了方便，我们重新定义自转移时的边界，在第一层调用 $f(n)$ 仍继续递归，认为第二层调用的 $f(n) = +\infty$ 。

记 $s = p \bmod 2k, t = p - \lfloor \frac{p+k}{2k} \rfloor$ 。

- 若 $s = k$ ，则 $f(p) = 1 \leq \max\{f(p-1), f(p+1)\}$;
- 若 $s = k + 1$ ，则 $f(p) = 1 + \min\{f(t+1), f(t)\} \leq \max\{1, 1 + \min\{\max\{f(t-1), f(t+1)\}, f(t)\}\} = \max\{f(p-1), f(p+1)\}$;
- 否则若 $s \neq k - 1$ ，则 $\lfloor \frac{p-1+k}{2k} \rfloor = \lfloor \frac{p+k}{2k} \rfloor = \lfloor \frac{p+1+k}{2k} \rfloor$ ，记 $a_1 = \min\{f(t-2), f(t-1)\}, a_2 = \min\{f(t-1), f(t)\}, a_3 = \min\{f(t), f(t+1)\}, a_4 = \min\{f(t+1), f(t+2)\}$ ，则 $f(p) = 1 + \max\{a_2, a_3\} \leq 1 + \max\{a_1, a_2, a_3, a_4\} = \max\{f(p-1), f(p+1)\}$ 。

因此这确实是对的。那接下来就可以用其进行优化。

发现当 $m \neq k + 1$ 且 $m \neq k$ 且 $s = k - 1$ 时, $f(p + 1) = 1$ 。
 所以改进后的转移如下:

$$f(n) = \begin{cases} 1, & m = k \\ 1 + \min\{f(p), f(p + 1)\}, & m = k + 1 \\ 1 + \min\{f(p), f(p - 1)\}, & m \neq k + 1 \text{ 且 } m \neq k \text{ 且 } s = k - 1 \\ 1 + f(p), & \text{otherwise} \end{cases}$$

发现当 $m \neq k+1$ 且 $m \neq k$ 且 $s = k-1$ 时, $f(p+1) = 1$ 。
所以改进后的转移如下:

$$f(n) = \begin{cases} 1, & m = k \\ 1 + \min\{f(p), f(p+1)\}, & m = k+1 \\ 1 + \min\{f(p), f(p-1)\}, & m \neq k+1 \text{ 且 } m \neq k \text{ 且 } s = k-1 \\ 1 + f(p), & \text{otherwise} \end{cases}$$

对于最后一种情况, 可以直接连续跳 $\max\{1, \lfloor \frac{(n+k) \bmod 2k}{x} \rfloor\}$ 步, 发现这几步中除 $x=1$ 外不会出现前三种情况, 而手模 $x=1$ 后发现仍然正确。

发现当 $m \neq k + 1$ 且 $m \neq k$ 且 $s = k - 1$ 时, $f(p + 1) = 1$ 。
所以改进后的转移如下:

$$f(n) = \begin{cases} 1, & m = k \\ 1 + \min\{f(p), f(p + 1)\}, & m = k + 1 \\ 1 + \min\{f(p), f(p - 1)\}, & m \neq k + 1 \text{ 且 } m \neq k \text{ 且 } s = k - 1 \\ 1 + f(p), & \text{otherwise} \end{cases}$$

对于最后一种情况, 可以直接连续跳 $\max\{1, \lfloor \frac{(n+k) \bmod 2k}{x} \rfloor\}$ 步, 发现这几步中除 $x = 1$ 外不会出现前三种情况, 而手模 $x = 1$ 后发现仍然正确。

因为 $m = k$ 时立刻返回 1, $m = k + 1$ 时会向后扩展, 遇到 $s = k - 1$ 会向前扩展, 所以递归每一层最多只有两个状态会被访问。

分析一下复杂度，把状态拆成 $\leq (2k)^2$ 和 $> (2k)^2$ 两类，处理前者的复杂度为 $O(\min\{\frac{n}{2k}, 2k\})$ ，后者的复杂度为 $O(2k(\ln n - 2 \ln(2k)))$ 。

分析一下复杂度，把状态拆成 $\leq (2k)^2$ 和 $> (2k)^2$ 两类，处理前者的复杂度为 $O(\min\{\frac{n}{2k}, 2k\})$ ，后者的复杂度为 $O(2k(\ln n - 2 \ln(2k)))$ 。

两者都不超过 $O(\sqrt{n})$ ，因此总时间复杂度 $O(\sum \sqrt{n})$ 。

Thanks!!!